SAII | PROJEKTARBEIT

Grober Ablaufplan
Kundenprojekt Zeiterfassung

Team: f_you_goethe **Abgabe:** Kurzplanung

1. Projektziel

Wir entwickeln eine Zeiterfassungsanwendung, stellen sie in einer Kubernetes-Kundenumgebung bereit und überwachen ihren Betrieb. Benutzer sollen sich anmelden sowie Arbeitsbeginn und Arbeitsende erfassen können. Ein Adminbereich zeigt die gespeicherten Zeiten an. Monitoring erkennt Probleme; relevante Alarme werden über Mattermost an das Betriebsteam gemeldet.

2. Grober Ablauf

Nr.	Arbeitspaket	Kurze Erklärung	Ergebnis
01	Anforderungen & Planung	Funktionen, Systeme, Zuständigkeiten und Arbeitsreihenfolge festlegen.	Geplanter Aufbau und Aufgabenverteilung
02	Git-Repository aufbauen	Repository erstellen, Zugriffe einrichten und eine einfache Projektstruktur für Code und Konfiguration anlegen.	Gemeinsames Git-Repository
03	LM1 vorbereiten	Docker/Minikube, lokale Anwendung und PostgreSQL für die Entwicklung einrichten.	Lauffähige lokale Umgebung
04	Zeiterfassung entwickeln	Login, Beginn/Ende der Arbeitszeit und eine einfache Adminansicht umsetzen.	Erste funktionsfähige Version
05	Lokal testen	Funktionen prüfen, Fehler korrigieren und auf Kali geeignete GUI-/Sicherheitstests durchführen.	Dokumentierte Testresultate
06	Push/Pull & CI aufbauen	Code zu Git pushen; auf den beteiligten LMs pullen. Automatisierte Builds und Tests konfigurieren.	Nachvollziehbare Änderungen und Pipeline
07	Images in Registry speichern	Container-Image erzeugen, versionieren und in die Registry pushen.	Abrufbares Container-Image
08	LM2 / Kubernetes bereitstellen	Kundenumgebung, Namespaces, App-Service, Ingress sowie Datenbank mit persistentem Speicher vorbereiten.	Vorbereitete Kundenumgebung
09	Deployment durchführen	Anwendung via Argo CD/GitOps auf LM2 ausrollen und Container-Image aus Registry abrufen.	Erreichbare Anwendung auf LM2
10	Kundenseitige Tests	Login, Stempelung, Speicherung und Adminansicht im Zielsystem prüfen.	Bestätigte Kernfunktionen
11	Monitoring einrichten	Prometheus sammelt Metriken, Grafana zeigt Zustände und ELK unterstützt die Logsuche.	Dashboards, Metriken und Logs
12	Alerting & Mattermost	Alertmanager-Regeln festlegen und Alarmmeldungen über einen Webhook an Mattermost senden.	Funktionierender Alarmkanal
13	Gesamttest & Übergabe	Kompletten Weg von Commit bis Alarm prüfen, Ergebnisse dokumentieren und Kundenseite abnehmen lassen.	Projektdokumentation / Abnahme

3. Technischer Ablauf – vom Commit bis zum Alarm

LM1 (Code entwickeln) → Git (Code und Konfiguration speichern) → CI (bauen und testen) → Registry (Images speichern) → Argo CD / Kubernetes auf LM2 (bereitstellen) → Monitoring (Metriken und Logs) → Alertmanager (Regeln auswerten) → Mattermost (Alarm melden).

Parallel zum Releasefluss greifen Benutzer über HTTPS auf die Zeiterfassung zu. Die Anwendung kommuniziert intern mit PostgreSQL; die Datenbank ist nicht direkt über das Internet erreichbar.

4. Beteiligte Systeme und ihre Aufgaben

System	Aufgabe
Kali	GUI-, Funktions- und Sicherheitstests
LM1	Lokale Entwicklung und Tests mit Docker/Minikube
Git	Quellcode und Konfigurationsdateien versionieren
CI / Registry	Builds, Tests und versionierte Container-Images
LM2 mit Kubernetes	Bereitstellung und Betrieb der Anwendung
PostgreSQL	Benutzer und Zeitbuchungen dauerhaft speichern
Prometheus / Grafana / ELK	Metriken sammeln, Dashboards anzeigen und Logs auswerten
Alertmanager / Mattermost	Warnungen erkennen und Betriebsteam benachrichtigen

5. Wichtige Schnittstellen (Planungswerte)

- HTTPS / TCP 443: Git-Zugriffe, Registry-Zugriff, Weboberfläche und gegebenenfalls Webhooks.
- TCP 5000: angenommener interner Port der Zeiterfassungsanwendung.
- TCP 5432: PostgreSQL, nur innerhalb der jeweiligen Umgebung erreichbar.
- Kubernetes-Service/Ingress: Zugang zur Anwendung; konkrete DNS-Namen und Freigaben werden später festgelegt.

6. Voraussetzungen, Risiken und offene Annahmen

- IP-Adressen, Hostnamen, Registry-Anbieter und Ports müssen an die tatsächliche Schul- bzw. Kundennetzumgebung angepasst werden.
- Die Reihenfolge setzt voraus, dass LM1, LM2, Git und nötige Zugriffsrechte verfügbar sind.
- Es wird angenommen, dass Argo CD, Prometheus, Grafana, ELK und Mattermost eingesetzt werden; konkrete Betriebsdetails sind noch zu klären.
- Der Plan ist bewusst grob. Einzelne Schritte können parallel ausgeführt und nach Tests angepasst werden.

7. Definition fertig / Abnahme

Das Projekt gilt als abgabebereit, wenn die Anwendung auf LM2 erreichbar ist, Login und Stempelung funktionieren, die Buchungen dauerhaft gespeichert werden, der Adminbereich Zeiten anzeigt, Monitoringdaten sichtbar sind und ein Testalarm in Mattermost ankommt. Zusätzlich liegen Git-Repository, Testnachweise und die Projektdokumentation vor.